

Question 1

- a) An *optimisation problem* is the task of finding the best solution from all feasible solutions. Provide **TWO (2)** examples of *optimisation problem*. (2 marks)

- University Course Scheduling Problem
- N-Queens Problem
- Sudoku
- Traveling-Salesman Problem

- b) Unlike path-searching problems that are usually addressed using *classical search algorithms*, *optimisation problems* can often be solved using *local search algorithms*. Differentiate between *local search algorithms* and *classical search algorithms* in **TWO (2)** ways. (4 marks)

Classical Search	Local Search
Searches through a search tree to find a path from the initial state to a goal state.	Starts with a current state and moves to a neighbouring state to improve the solution.
Usually considers the path to the solution as important.	Mainly focuses on the quality of the final state, not the path taken.
Examples: BFS, DFS, Uniform Cost Search, A*.	Examples: Hill Climbing, Simulated Annealing, Local Beam Search.
Often requires more memory to store search nodes/frontier.	Usually uses less memory because it stores only a small number of states.
Return a sequence of actions as a solution	Return a single final state as a solution

- c) *Hill climbing* is a well-known *local search algorithm*.

- (i) Briefly describe **TWO (2)** advantages of using it.

(2 marks)

- **Low memory usage** – It only stores the current state and neighbouring states, so it requires little memory.
- **Fast and simple (Greedy)** – It continuously moves to a better neighbouring state, making it simple and often fast for large search spaces.

(ii) Briefly describe **TWO (2)** disadvantages of using it.

(2 marks)

- **Not complete** – It may fail to find a solution even when one exists.
- **Not optimal** – It does not guarantee finding the best solution.
- **Sensitive to initial state** – Different starting points can lead to different solutions.
- **Can get stuck at a local maximum** – It may stop at a solution that is better than its neighbours but not the best overall solution.

(iii) Provide **TWO (2)** other examples of *local search algorithms*.

(2 marks)

- Simulated Annealing
- Local Beam Search
- Genetic Algorithm

d) A *state-space landscape* is a graphical representation of the states of a problem reachable by a search algorithm, which is usually used to guide the search by visualising how solutions are distributed and how they can be improved.

- (i) *Current state, global maximum, local maximum, flat local maximum, and shoulder* are **FIVE (5)** important elements commonly found in a *state-space landscape*. Briefly describe each of them. (5 marks)

Current state

- The state where the agent is **currently** located at.
- **Example:** Imagine standing on a spot of the mountain — that's your current position.

Global maximum

- The best possible state with the **highest value** of objective function.
- **Example:** If the agent reaches the top of Mount Everest, it has found the best possible solution.

Local maximum

- A state whose value is **higher** than all of its neighboring states, but **not the best one**.
- **Example:** The agent might think it's done, but it's stuck at a "smaller hill."

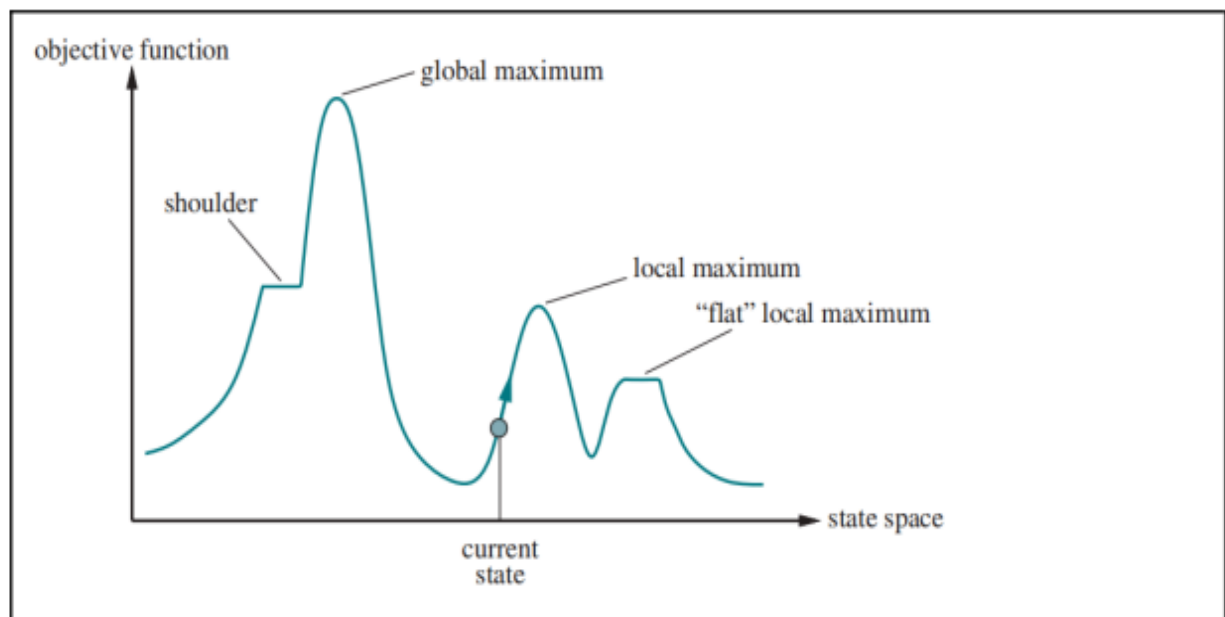
Flat local maximum

- A region where the current state and its neighboring states have the **same objective-function value**.
- **Example:** Standing on a flat area where every direction has the same height.

Shoulder

- A plateau whose edge is **stretching upwards** (going in the same overall direction).
- **Example:** You are walking across a flat area. At first, every direction seems equally high, but eventually there is an upward slope leading to a higher peak.

- (ii) Sketch **ONE (1)** *state-space landscape* that includes the **FIVE (5)** elements stated in **Question 1 d) (i)**, with elevation defined by the values of an *objective function*. You are required to label the X-axis, Y-axis, and all the **FIVE (5)** elements. (8 marks)



Question 2

- a) Most definitions of Artificial Intelligence (AI) can be categorised along two dimensions: *human performance* vs. *rationality* and *thought processes* vs. *behaviour*, resulting in four approaches to AI development. Identify these **FOUR (4)** approaches. (4 marks)

- Cognitive Modelling Approach (Thinking Humanly)
- Law of Thought Approach (Thinking Rationally)
- Turing Test Approach (Acting Humanly)
- Rational Agent Approach (Acting Rationally)

- b) One of the four approaches stated in **Question 2 a)** involves designing and developing agents that perceive their environments through sensors and act upon the environments through actuators. Such agents typically interact with their environments in two ways. Identify the **TWO (2)** types of interaction. Then, differentiate between them. (2 + 2 marks)

Interaction	Explanation
Perception	The agent receives information from the environment through sensors and uses it to understand the current state of the environment.
Action	The agent changes or affects the environment through actuators based on the information it has perceived and its decision.

- c) Suppose an autonomous driving car named *SmartCar* is an example agent stated in **Question 2 b)**. The problems that *SmartCar* aims to solve can be described using the *PEAS* framework.
- (i) State what *PEAS* stands for. (2 marks)

P: Performance Measure
E: Environment

A: Actuator
S: Sensor

(ii) Outline a *PEAS* description for *SmartCar*'s tasks.

(4 marks)

Performance Measure	Safety, reaching the destination, obeying traffic rules, minimising travel time and fuel consumption
Environment	Roads, traffic lights, other vehicles, pedestrians, weather, road conditions
Actuator	Steering, accelerator, brakes, horn, indicators
Sensor	Camera, GPS, LiDAR, radar, ultrasonic sensors

- d) Suppose *TAR-GPS* is a GPS system designed by TAR UMT students and installed in *SmartCar* to find the shortest path from Station A to Station H, as shown in **Figure 1**. **Figure 1** illustrates a directed graph of eight stations with edge weights representing distances (km) between stations, while **Table 1** lists the heuristic cost of each station relative to Station H.

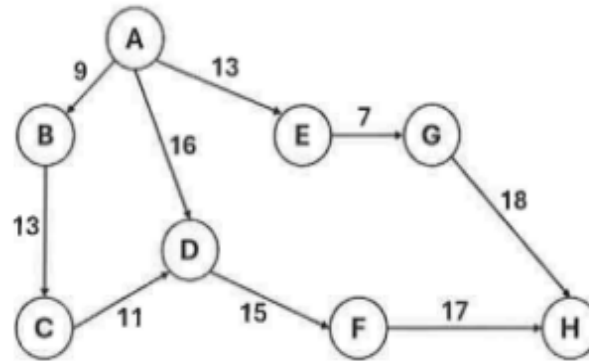


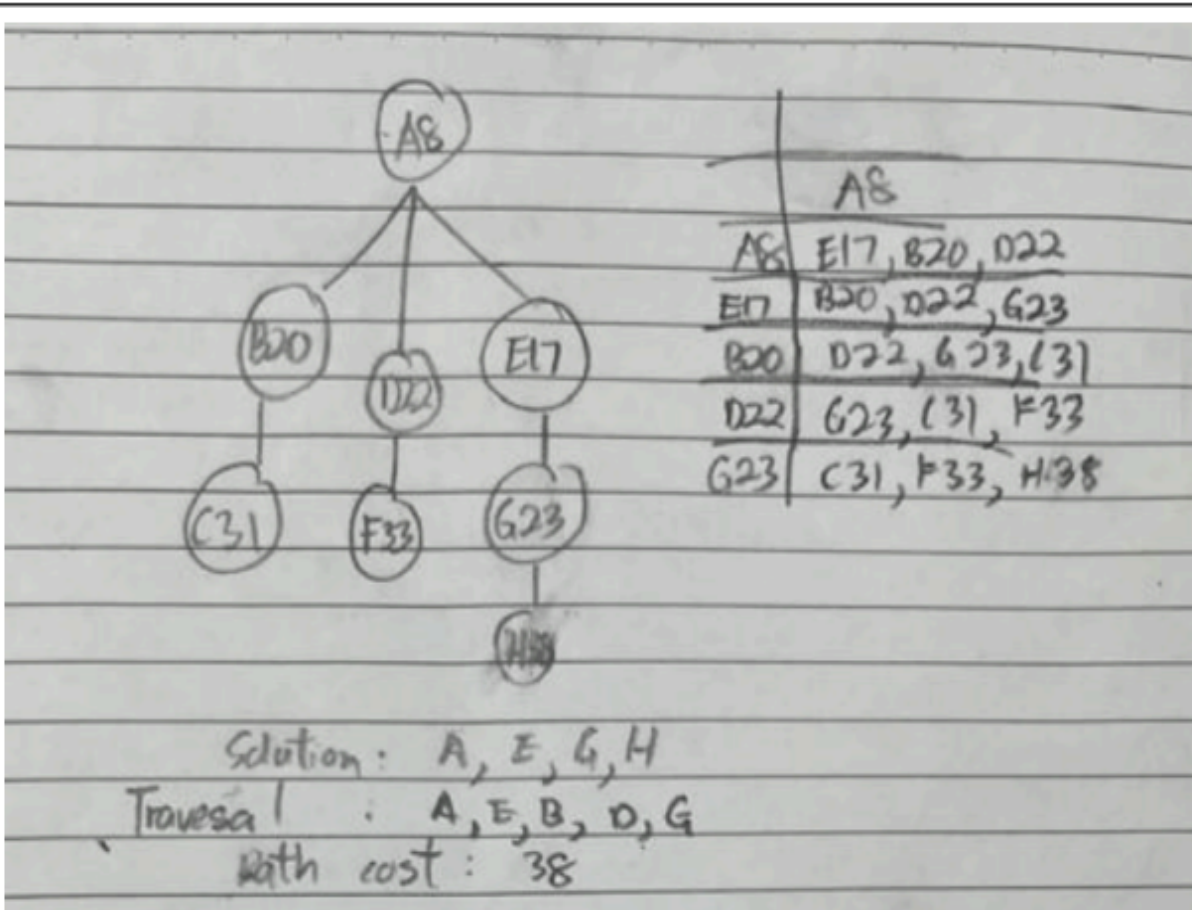
Figure 1. A map with eight stations.

Table 1: Heuristic cost of each station.

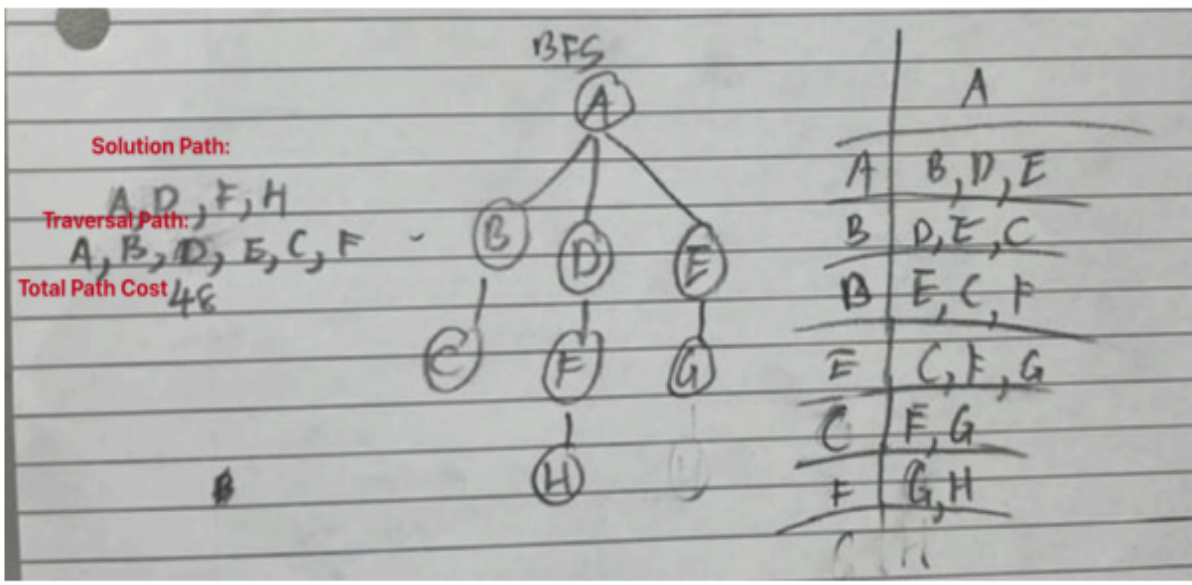
Station	A	B	C	D	E	F	G	H
Euclidean distance	8	11	9	6	4	2	3	0

- Based on the information provided in **Figure 1** and **Table 1**, illustrate how the A^* search algorithm searches for a path to Station H starting from Station A by drawing **ONE (1)** search tree, assuming nodes are expanded in alphabetical order (if applicable). (8 marks)
- Identify the *search path* explored and the *solution path* returned. (2 marks)
- Calculate the *total cost* of the *solution path* returned. (1 mark)

A^*

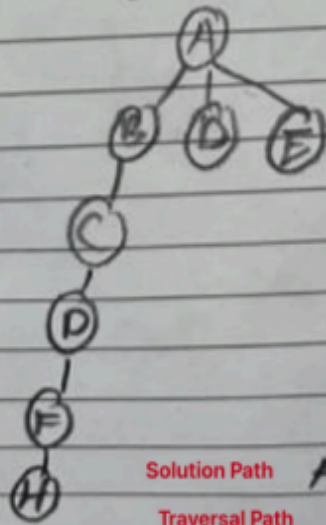


BFS



DFS

DFS



	A
A	B, D, E
B	C, D, E
C	D, D, E
D	F, D, E
F	H, D, E

Solution Path A, B, C, D, F, H

Traversal Path A, B, C, D, F

Total Path Cost 65

Question 3

- a) The commonly studied games in AI usually share certain characteristics. Identify **FIVE (5)** of these characteristics. (5 marks)

Characteristics	Description
Two-player	A game that is played between two players.
Turn-taking	Players take turns making moves until the game ends.
Perfect information	Both players can see the complete state of the game (Fully Observable).
Deterministic	The same action always produces the same result; there is no randomness.
Zero-sum	One player's gain is the other player's loss. There is no win-win outcome.

- b) Answer the following questions based on the simple game tree provided in **Figure 2**.

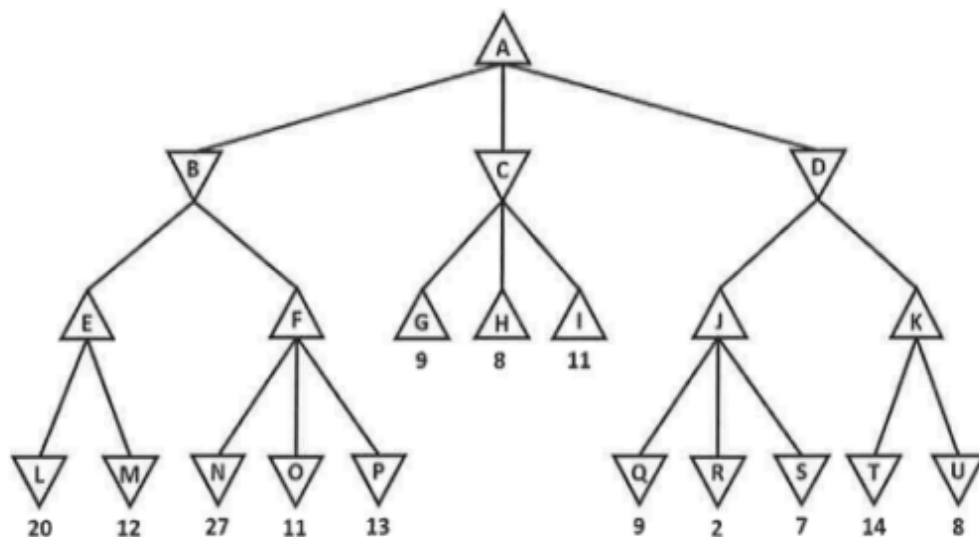


Figure 2. A simple game tree

- (i) Identify the *minimax* values for **EIGHT (8)** nodes: A to F, J and K. (4 marks)

- (ii) Identify the *best move* at the root node. Justify your answer. (1 + 1 marks)

- (iii) Apply minimax search with *alpha-beta pruning* in *left-to-right order* and identify nodes that would be pruned. Then, apply minimax search with *alpha-beta pruning* in *right-to-left order* and identify nodes that would be pruned. For each order, you must demonstrate how *alpha-beta pruning* works on the game tree provided in **Figure 2**. (12 marks)

- (iv) Derive a conclusion based on your answer to **Question 3 b) (iii)**. (2 marks)

Question 4

- a) A *Constraint Satisfaction Problem (CSP)* consists of variables, domains, and constraints. Solving a *CSP* therefore involves assigning values to variables while satisfying all specified constraints. There are several types of assignments, including *consistent*, *legal*, *partial*, and *complete* assignments.

- (i) Differentiate between these **FOUR (4)** types of assignments. (4 marks)

Consistent/legal assignment

- An assignment that does not violate any constraints.

Complete assignment

- An assignment where all variables have values.

Partial assignment

- An assignment where only some variables have values.

Partial Solution

- A consistent partial assignment that does not violate any constraints.

- (ii) Define a *solution* to a *CSP*. (2 marks)

- A solution is an assignment of values to all variables.
- The assigned values must satisfy all the constraints.
- In simple words: every variable has a value, and there are no constraint violations.

- b) Sudoku is a puzzle game that involves the logical placement of numbers and has introduced millions of people to the concept of *CSP*. **Figure 3** shows an example of a 4×4 Sudoku grid consisting of 16 cells, some of which are pre-filled with digits from 2 to 4. The objective is to fill the grid such that each row, column, and 2×2 sub-grid contains each digit—from 1 to 4—exactly once. Answer the following two questions.

	1	2	3	4
A		2	3	4
B	3			2
C	2			3
D			2	

Figure 3. State x of the 4×4 Sudoku.

- (i) Suppose the *AllDiff* global constraint is applied to solve the Sudoku puzzle in **Figure 3**. Analyse the Sudoku puzzle as a *CSP* by clearly identifying the *variables*, the *domain* of each variable, and the *constraints*, expressed using the *AllDiff* constraint. (9 marks)

Variable, $X = \{A1, \dots A4, B1, \dots B4, C1, \dots C4, D1, \dots D4\}$

Domain of empty squares, $D_i = \{1, 2, 3, 4\}$.

C =

row

Alldiff (A1, A2, A3, A4)

Alldiff (B1, B2, B3, B4)

Alldiff (C1, C2, C3, C4)

Alldiff (D1, D2, D3, D4)

subgrids

Alldiff (A1, A2, B1, B2)

Alldiff (A3, A4, B3, B4)

Alldiff (C1, C2, D1, D2)

Alldiff (C3, C4, D3, D4)

Column

Alldiff (A1, B1, C1, D1)

Alldiff (A2, B2, C2, D2)

Alldiff (A3, B3, C3, D3)

Alldiff (A4, B4, C4, D4)

- (ii) To simplify implementation, suppose all *AllDiff* constraints are expanded into binary constraints, allowing the *AC-3* algorithm—an algorithm for enforcing arc consistency—to be applied directly. Then, explain, step by step, how the values of A1, B2, B3, C3, and D4 are determined in sequence, by specifying which constraint is applied at each step and how it leads to the value assignment. (10 marks)

A1

- From the constraints in the row, we can remove 2, 3, and 4 from A1's domain. As a result, the domain is reduced to {1}.

B2

- From the constraints in the box, we can remove 1, 2, and 3 from B2's domain. As a result, the domain is reduced to {4}.

B3

- From the constraints in the row, we can remove 2, 3, and 4 from B3's domain. As a result, the domain is reduced to {1}.

C3

- From the constraints in the column, we can remove 1, 2, and 3 from A1's domain. As a result, the domain is reduced to {4}.